Autonomous Design Report

EPFL Racing Team

Louis Gounot, Mattéo Berthet, Vincent Philippoz, Joe Najm, Maximilian Gangloff, Baudoin Bosc, Tudor Oancea, Philippe Servant, Antonio Pisanello, Francesco Nonis, Alexandre Vray

June 2023

Event	**Target points**	Maximum points	Ranking
Acceleration	**3.5**	75	Top 31.6 %
Skidpad	**3.5**	75	Top 47.4 %
Autocross	**42**	100	Top 26.3 %
Trackdrive	**15**	200	Top 26.3 %
Total	**64**	450	Top 36.8 %

Table 1: Target points for 2023 season. These are the points that would be obtained in 2022 when finishing each event. It correspond to what the last team who successfully finished the event scored.

1 Introduction

1.1 Context

The EPFL Racing Team is gearing up to compete with its fourth car in the upcoming summer of 2023. Although this season will be the first one in which we compete in the Driverless categories, the team has been working on its autonomous stack for almost two years. The first year mainly consisted in the development of the algorithms in simulation and deploying them at a smaller scale on a 1:8th RC car. With the use of just one camera, it was able to successfully navigate skidpads and acceleration tracks. This year, we optimized the performance and robustness of the algorithms, and were able to deploy them on our first actual Driverless-capable car. Having already gained experience and perspective on the subject, we are determined to successfully complete the four events: acceleration, skidpad, autocross and trackdrive.

1.2 Design objectives

As this is the first autonomous vehicle, the main objective is to finish each event successfully. Looking at the 2022 results, 64 points can be obtained, as shown by table 1. To do so, reliability is the first priority. *To finish first, first you have to finish.* All systems and software are conceived to be fail-safe and as robust as possible.

2 Architecture

The general organization of our autonomous system can be found in figure 1. The computations are divided between two computing units: an Nvidia Jetson running the high-level autonomous stack and an sbRIO running the low-level control and safety protocols (see 3.4).

All the code on the Jetson was implemented in Python and C++ nodes using ROS 2 Humble, unlike most of the other teams that use ROS 1 Noetic. We made this choice because Noetic will enter EOL in less than two years, and we want to avoid migration of the codebase that will have probably grown a lot by then. Moreover, all packages that we use are already ported to ROS 2, so it is not a limitation.

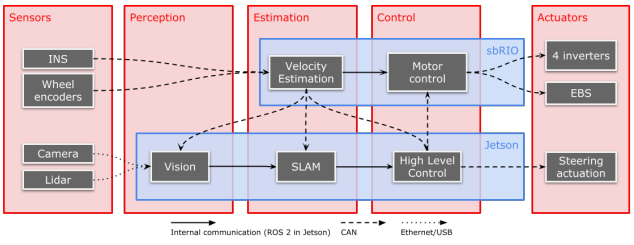

Figure 1: High-level diagram of the autonomous system

3 Hardware

3.1 Steering system

The autonomous steering system was primarily designed with reliability and (angular) precision in mind, although secondary priorities were energy consumption and weight. Our goal was for the system to be able to travel a full steering range in less than 1.5 seconds, which is the speed that an actual pilot does during skidpad.

The actuation is done by a stepper motor connected to a gearbox (ratio of 5.09) and a 20 000 steps encoder, powered by 48V. It produces an equivalent to 21 Nm torque at the steering wheel. A high safety factor is used, as necessary for our first year in the driverless category. While DC motors are a better option performance-wise, they can't work in open-loop. We wanted to keep this option open in case closed-loop control wasn't working.

To ensure angular precision, we designed a mechanical system that would limit any play between parts. Thus, the system is directly connected to one of the pinions used to actuate the custom double rack. When engaged, the system transmits torque to the rack thanks to a precisely machined motor-pinion link. Using an electromagnetic clutch was considered but discarded, as it is too heavy, bulky, and power-hungry. Thus, the disconnection is a purely mechanical operation: the motor slides back (without detaching the whole motor assembly) to clear any contact between the exit of the gearbox and the bridge piece.

3.2 Braking system

The standard autonomous braking of the car is achieved by regeneration. The EBS is implemented using a pneumatic system. Two identical instances of the EBS are implemented to ensure redundancy: one acts on the front brakes and the other on the rear brakes. The objective of the EBS is to reach a mean deceleration of 8 m/s and to allow 6 firings per reload of the air tanks. 6 reloads are enough to be safe in any case, and it is not necessary to oversize the system to have more firings.

The implemented system uses 0.4 L air tanks filled up to a pressure of 14 bars. A pressure regulator brings the pressure down to 6 bars. The volume of the tank, their filling pressure and the operating pressure of the system were chosen jointly with the objective of keeping the filling pressure as low as possible for safety, while ensuring the 6 firings and the target deceleration. Manual valves allow to arm/disarm the system. Solenoid valves are used to trigger the system. They are normally open, meaning that no voltage supply is required to allow the air to flow through them. Thus, the EBS is triggered even in the absence of voltage supply, making it fully fail-safe. Some pneumatic cylinders actuate some master cylinder, which will then actuate the brakes through the braking lines. Pressure intensifiers to convert the pneumatic pressure into hydraulic pressure were considered. However, those components must be custom-made and present a higher risk of leaks if not designed appropriately. As this is the first EBS that will be implemented by our team, it was preferred to discard this option to confirm and fix the rest of the design. The use of pressure intensifiers will however be a strong optimization point for future cars, as it will allow reducing the weight and size of the system. Finally, the EBS brake lines are merged with the brake pedal ones by means of a shuttle valve.

3.3 Sensors

Two sensors are used for the perception: A camera and a LiDAR. The LiDAR captures a 360°x45° point cloud. With 64 channels, the resolution is 0.7° and allows a good detection of the cones. The LiDAR provides precise ranging information for every cone, information that is hard to infer precisely from a camera. One monocular camera is used to recognize cones and assess their color. The camera can capture up to 47 4112x2176 px color images per second. The 6mm F1.8 lens provides a wide FOV (96°H, 79°V). Both perception sensors are mounted at the highest point of the car, as it is the place that offers the greatest visibility. This spot also mitigates cones hiding other cones. The fixation has 0° roll and 0° pitch, as it was found to be the best setting experimentally.

Finally, a GNSS system is integrated to aid the localization and mapping algorithms. RTK technology is used to achieve centimeter level precision in position. Velocity is also measured accurately. Furthermore, the setup uses two antennas, therefore the yaw and roll angles can be estimated precisely. This helps a lot in the localization algorithms, both during initialization and at run time.

3.4 Computing platform

The driverless algorithms run on an Nvidia Jetson Orin. This computing platform offers a good compromise between computing performance and energy consumption. Also, the unit is compact and easy to interface with its numerous ports. This computer is dedicated to the autonomous system. The rest of the software which controls the motors, read sensors is taken care of by a sbRIO board, as it is more robust in harsh

environments. A CAN bus allows communication between the two computing platforms and the other peripherals.

4 Vision

The vision aims to detect cones and estimate their distances to the car at 10Hz frequency. We chose 10Hz over 20Hz for real-time implementation, allowing flexibility for potential algorithm delays or communication issues via ROS. The primary vision pipeline combines camera and LiDAR sensor data, with backup options of using only LiDAR or camera in case of sensor failure or message loss. These alternatives are activated dynamically during a run if any sensor malfunctions.

4.1 Synchronization between Camera and LiDAR

Our main vision algorithm combines camera and LiDAR data, requiring synchronization to minimize acquisition time differences. The LiDAR serves as the primary vision clock and runs at 20 Hz. However, it only publishes every second frame, thus publishing in reality at 10 Hz. This is to reduce delays between image and point cloud and the time between data acquisition and publishing the data. When the LiDAR begins its acquisition, it sends an empty ROS message to notify the camera and vision node of the start of a new frame. The camera node then captures and publishes an image while the LiDAR is still acquiring its frame. This allows the vision node to process the image before receiving the point cloud.

4.2 Camera-only algorithm

Cones are detected and classified using YOLOv7, a reliable and real-time detection algorithm chosen for its state-of-the-art status at the start of pipeline implementation in September 2022. Besides the advantages of the YOLO series, YOLOv7 also supports pose estimation, which was modified to get directly all the bounding boxes and keypoints of the cones. Leveraging the known geometry of the cones, a PnP algorithm extracts translation vectors, providing distance information to the car.

4.3 Lidar-only algorithms

The LiDAR-only algorithm utilizes a two-step process. Firstly, a ground-removal step is performed, considering the potential non-flatness of the ground. Next, a clustering technique, specifically DBSCAN, is applied to the resulting point cloud to distinguish cones. The clustering considers both cluster size and maximum point distances to differentiate cones from other objects present near the track, such as walls or other cars. Two approaches were considered for ground removal: RANSAC-based and Ground Bins.

4.3.1 RANSAC

This algorithm first randomly chooses three points from the point cloud and fits a plane between these points. Then, all the points within a threshold of the fitted plane are considered as being part of the plane. This is repeated 50 times, thus the biggest of all the planes should be the ground. Furthermore, a bin-based RANSAC algorithm is also implemented, where the point cloud is split into 3x3 m^2 bins and RANSAC is done on each bin, in order to take into account the possibility that the ground is not completely flat.

4.3.2 Ground bins

We implemented a ground bins algorithm as an alternative approach. The point cloud is divided into small 1.1x1.1 m^2 bins, and the lowest and highest z-coordinates are obtained within each bin. Bins with a point below a specified threshold and a significant difference between z_{max} and z_{min} are identified as potentially containing objects other than the ground. Points above $z_{min} + \delta$ (with δ as a threshold) in these bins are classified as non-ground and extracted.

4.4 Sensor Fusion

Sensor fusion is the primary vision algorithm, projecting the LiDAR's point cloud onto the image and extracting points within the cones' bounding boxes for distance information and cone classification. Prior to projection, a RANSAC algorithm removes the ground from the point cloud. The resulting point cloud is converted to 2D and fused with the camera image using estimated extrinsic and intrinsic parameters. Points within each bounding box are extracted and subjected to DBSCAN clustering, isolating the largest cluster (the cone) while eliminating additional noise from surrounding objects within the bounding box.

4.5 Dynamic switching

Only one vision pipeline runs at a time. By default, sensor fusion is used, but in the event of sensor failure or missed messages, the system dynamically switches to the pipeline involving the functioning sensor. A watchdog is implemented to ensure that a new point cloud is received every 100ms. If no point cloud is received for two consecutive iterations (i.e. up to 200ms), the vision

node replaces the LiDAR as the vision clock until new point clouds are received. In such cases, the system switches from fusion to camera-only mode. To switch between fusion and LiDAR-only, the system checks if a new image has been obtained after receiving the point cloud.

5 Estimation

There are three main components that we have to estimate to allow the motion planning and control algorithms to properly follow the track: estimating the car's position, velocity, and the actual track's characteristics if it is not known beforehand.

5.1 Velocity estimation

The velocity estimation is done in parallel of the other state estimation as slips ratios are normal forces. To do so, a Mixed Kalman Filter has been implemented including a Linear Kalman Filter (LKF), an Extended Kalman Filter (EKF) and an Unscented Kalman Filter (UKF) in order to have a well covariance propagation for high nonlinear relations between state and measures.

The used sensors are the following :

- Motor speed encoder on each wheel : $\tilde{\omega}_i = \omega_i + \eta_\omega$
- Motor torques on each wheel : $\tilde{\tau}_i = \tau_i + \eta_\tau$
- Brake pressure front and rear : $\tilde{P}_{b,i} = P_{b,i} + \eta_{P_b}$
- accelerometers and yaw rate from INS.

As the accelerometers are known to be biased, the biases are removed each time the vehicle is stopped. The same strategy is used to update the lateral velocity which diverge without truly velocity sensor.

Each sensor values are considered at a frequency of 100 Hz. An additional velocity sensor RTK with a refresh frequency of 5Hz will also be added to the Kalman filter in order to have a better estimation of the states.

5.2 Localization & mapping

The localization and mapping tasks have two variants:

- *Localization-only* on tracks (i.e. their cones' positions) that are known beforehand. This corresponds to the acceleration, skidpad and trackdrive events.
- *Simultaneous Localization and Mapping (SLAM)* on tracks that are not known and have to be mapped while the car is driving. This corresponds to the Autocross event.

Both tasks are accomplished using an EKF fusing the cones observations and the velocity estimation, although using slightly different formulations. Both are classical algorithms that can be found in both academic references and other Formula Student software stacks. These two variations are:

1. The *EKF Localization*, which only considers the car pose as the state variable of the Kalman filter, and considers the positions of the landmarks as fixed.
2. The *EKF SLAM*, which also considers the landmarks positions in the state, and whose state dimension changes as landmarks are discovered and added to the map.

In both cases, the data association between the observed cones and the mapped landmarks is done by transforming all positions in global Cartesian coordinates, computing a distance matrix between the two sets of points using the Euclidean distance, and finally finding the optimal assignment using a linear assignment problem. Thresholds are applied on the associated pairs of cones/landmarks to discard spurious observations or declare observations as new landmarks in SLAM mode.

The SLAM algorithm also keeps track of a *health score* and *expected health score* for each mapped landmark that is used for feature management. At each iteration of the EKF SLAM, the expected health score of each landmark in a certain range (considered as observable by the vision module) is incremented by 1, and the health score is incremented by a score between 0 and 1 (penalized by the uncertainty of the observation) for each actually observed landmark. Then we remove from the map the landmarks that have a health/expected health ratio below a certain threshold. This allows us to remove false positives and uncertain landmarks.

5.3 Boundary estimation & center-line extraction

To estimate the boundaries of the track, we start by choosing an initial cone (the nearest to the position of the car) and then, using the class probability as well as the angle between each cone, the initial cone and the current direction, a score is computed for each cone to be likely in this boundary, that means being in the neighborhood of the initial cone. The cone with the lowest score is chosen to be added to the current boundary. The same is then repeated iteratively, with

	Acceleration	Skidpad	Autocross	Trackdrive
Track known beforehand	Yes	Yes	No	Yes
Curvature minimization	No	No	No	Yes
Control method	NMPC	NMPC	PI+Stanley	NMPC

Table 2: Summary of the motion planning and control strategies by event

the next cone being considered as the initial cone and the direction updated to the vector between the previous and the new cone. Once the score reaches a certain threshold, we switch to the other side of the boundary and repeat the process once again.

Finally, the center line is estimated by iterating over one boundary, for each point on the boundary we find the closest one on the other, taking the middle between these two points gives one point of the center line. This allows to be robust on cone misclassification as well as offset on the cone position.

6 Motion planning & control

While several approaches combine the motion planning and control tasks, we have made the choice to treat them separately to simplify the tuning procedure of the controllers. Another important simplification we made is to separate the motor control into a high-level controller that only computes a global torque command, and a low-level controller that finally computes the torque distribution between the four wheels using a torque-vectoring algorithm that will not be detailed here.

The implemented methods change depending on whether we know the track in advance (Acceleration, Skidpad and Trackdrive events) or not (Autocross event) and are summed up in table 2.

6.1 Motion planning

The center-line extracted by the previous module is used as a reference trajectory for all events but the trackdrive, where we perform a curvature minimization considering the track widths to approximate a time optimal trajectory. This reference trajectory is represented as cubic splines and its curvature is used to compute a reference velocity profile based on maximal longitudinal velocity and lateral acceleration, using the following formula:

$$a_y = \frac{v_x^2}{R} = \kappa v_x^2 \implies v_x^{\text{ref}} = \min\left(v_{x,\max}, \sqrt{\frac{a_{y,\max}}{\kappa}}\right).$$

where a_y is the lateral acceleration, v_x is the longitudinal velocity, and R, κ are respectively the corner radius and curvature.

6.2 Control

The most primitive method separates the longitudinal and lateral control into a PI controller and a Stanley controller (as introduced by Stanford University in the 2005 DARPA Grand Challenge). This combination has the advantage of being easy to tune and having negligible runtimes (< 0.5ms), and allowing us to run them at 100Hz (the limit given by the frequency of the sbRIO). However, its lack of anticipation of future situations (e.g. a) leads to suboptimal behavior at speeds higher than 5m/s.

To remedy this limitation, we also developed a Nonlinear Model Predictive Control (NMPC) method that tracks the given trajectory, yaw and velocity references given by the motion planning. We used a 2s prediction horizon, 4 DoF kinematic bicycle model that is easy to tune and still yields perfect reference tracking up to 10m/s, control rate constraints and penalties, and soft constraints on the velocity.

The well-known ACADOS open-source library was used to implement the NMPC using the Real-Time-Iteration algorithm and the HPIPHM quadratic program solver. The high performance of these libraries allows us to solve our NMPC problem in ≈ 5ms. The controller is however deployed at 20Hz to limit the CPU usage.

We can't however use NMPC during the autocross event because the boundary estimation procedure does not provide enough track look ahead to deploy NMPC with a long enough horizon.